

Preguntas de Programación Web

400 preguntas de opción múltiple sobre los apuntes 02 a 22, organizadas en 20 temas.

Cómo usar este documento. Lee cada pregunta e intentá responderla antes de mirar las opciones. Después elegí la que te parezca correcta.

Las respuestas y las explicaciones están en la app Fichero de Programación Web, que además te indica qué repasar.

El índice de la página siguiente lleva directo a cada tema.

Índice

1	Variables, alcance y expresiones	1–13
2	El tipo <code>boolean</code>	14–28
3	Los tipos <code>number</code> y <code>bigint</code>	29–43
4	El tipo <code>string</code> y Unicode	44–58
5	<code>undefined</code> , <code>null</code> y la ausencia de datos	59–71
6	Conversión y coerción de tipos	72–86
7	El tipo <code>symbol</code> y los protocolos de JavaScript	87–101
8	Arrays y matrices	102–125
9	Objetos, propiedades y referencias	126–147
10	<code>Map</code> , <code>Set</code> y colecciones especializadas	148–162
11	Estructuras de control	163–182
12	Errores y excepciones	183–200
13	Funciones	201–226
14	Programación funcional y pipelines de datos	227–246
15	Recursividad y árboles binarios	247–268
16	Programación Asíncrona	269–281
17	Gestión de archivos con Node.js	282–312
18	Desarrollo web: del documento a una aplicación del clima	313–362
19	Arrow.js: del contador al inventario	363–374
20	Cómo consumir una API REST	375–400

Variables, alcance y expresiones

Preguntas 1 a 13

[Volver al índice](#)

1 ¿Qué nombre de variable describe mejor la cantidad de días que faltan para un vencimiento?

- A diasHastaVencimiento
- B fechaDeVencimiento
- C diasTranscurridosDelPlazo

2 Después de ejecutar `let total;`, ¿qué valor tiene `total`?

- A null
- B ReferenceError
- C undefined

3 ¿Qué hace la sentencia `const iva = 0.21;`?

- A Declara iva sin valor
- B Declara e inicializa iva
- C Reasigna una constante anterior

4 Una variable acumula un total que cambia en cada paso de un cálculo. ¿Con qué palabra conviene declararla?

- A** `const`
 - B** `class`
 - C** `let`
-

5 Después de `const pedido = { estado: "nuevo" };`, ¿qué operación está permitida?

- A** Reasignar `pedido` a un objeto nuevo
 - B** Modificar `pedido.estado`, porque `const` fija solo la variable
 - C** Nada: el objeto queda inmutable
-

6 Una variable se declara con `var` dentro de un `if`, y ese `if` está dentro de una función. ¿Hasta dónde llega el alcance de la variable?

- A** A toda la función
 - B** Solo al bloque `if`
 - C** Al módulo que contiene la función
-

7 ¿Qué formas de declarar variables quedan limitadas al bloque de llaves de un `if` ?

- A** `let` y `const`
 - B** `var` y `let`
 - C** `var`, `let` y `const`
-

8 Fuera de un bloque existe una variable `estado` con el valor `"global"`. Dentro del bloque se declara `const estado = "local"`. ¿Qué valor tiene `estado` dentro del bloque?

- A** El estado global
 - B** `undefined`
 - C** El estado local
-

9 Dentro de un bloque, se lee una variable antes de la línea `const` que la declara. ¿Qué ocurre?

- A** Se obtiene `undefined`
 - B** Se produce `ReferenceError`
 - C** Se obtiene su valor futuro
-

10 ¿Qué muestra en la consola este código?

```
console.log(total);  
var total = 100;
```

- A** undefined
 - B** 100
 - C** ReferenceError
-

11 Una función exterior declara una variable local y devuelve una función interior que usa esa variable. ¿Qué pasa con la variable cuando la función exterior termina?

- A** El dato se libera al terminar la primera función
 - B** La función devuelta recibe una copia congelada del dato
 - C** La función devuelta sigue accediendo a esa variable local
-

12 ¿Cuál de estos fragmentos es una expresión, es decir, produce un valor?

- A** `if (edad >= 18) {}`
 - B** `edad >= 18`
 - C** `const edad = 18;`
-

13 Después de ejecutar `let n = 5; const previo = n++;`, ¿cuánto valen `previo` y `n`?

- A** previo vale 5 y n vale 6
 - B** previo vale 6 y n vale 6
 - C** previo vale 5 y n vale 5
-

El tipo boolean

Preguntas 14 a 28

[Volver al índice](#)

14 ¿Cuáles son los únicos valores del tipo boolean?

- A** 1 y 0
 - B** truthy y falsy
 - C** true y false
-

15 ¿Puede un string no vacío usarse directamente como condición en un `if` ?

- A** No, se exige un booleano literal
 - B** Sí, se interpreta como truthy
 - C** Sí, pero se interpreta como false
-

16 ¿Cuál de estos valores es falsy?

- A** `" "`
 - B** `"false"`
 - C** `NaN`
-

17 ¿Qué devuelve `Boolean({})` ?

- A** false
 - B** true
 - C** undefined
-

18 ¿Qué devuelve `Boolean("false")` ?

- A** true, porque el string no está vacío
 - B** false, porque el texto dice false
 - C** Lanza `TypeError`, porque no es un booleano válido
-

19 ¿Qué expresión convierte `valor` a booleano de la forma más explícita y clara?

- A** `valor == true`
 - B** `Boolean(valor)`
 - C** `new Boolean(valor)`
-

20 ¿Qué devuelve `!0` ?

- A** false
 - B** 0
 - C** true
-

21 ¿Qué devuelve `"" && ejecutar()` ?

- A** El string vacío sin llamar a ejecutar
 - B** El resultado de ejecutar
 - C** El booleano false después de llamar a ejecutar
-

22 ¿Qué devuelve `"Ana" && 20` ?

- A** "Ana"
 - B** 20
 - C** true
-

23 ¿Qué devuelve `0 || false || null` ?

- A** false
 - B** true
 - C** null
-

24 ¿Qué devuelve `0 ?? 1` ?

- A** 0
 - B** 1
 - C** null
-

25 ¿Qué devuelve `5 === "5"` ?

- A** `false`
 - B** `true`
 - C** `undefined`
-

26 ¿Qué expresión comprueba correctamente si `valor` es `NaN` ?

- A** `valor === NaN`
 - B** `typeof valor === "NaN"`
 - C** `Number.isNaN(valor)`
-

27 ¿Qué expresión equivale a `!(a && b)` ?

- A** `!a && !b`
 - B** `!a || !b`
 - C** `a || b`
-

28 ¿Cómo evalúa JavaScript la expresión `esAdmin || esEditor && estaActivo` ?

- A** `(esAdmin || esEditor) && estaActivo`
 - B** `esAdmin || (esEditor && estaActivo)`
 - C** `!(esAdmin || esEditor) && estaActivo`
-

Los tipos `number` y `bigint`

Preguntas 29 a 43

[Volver al índice](#)

29 ¿Qué tipo representa tanto 42 como 3.5 en JavaScript?

- A** `number`
 - B** `bigint`
 - C** `integer`
-

30 ¿Qué valor decimal representa `0b101010` ?

- A** 101010
 - B** 52
 - C** 42
-

31 ¿Qué valor tiene `46_000_000` ?

- A** 46000
 - B** `SyntaxError`
 - C** 46000000
-

32 ¿Qué devuelve `-5 % 3` en JavaScript?

- A** -2
 - B** 1
 - C** NaN
-

33 ¿Qué resultado da `1 / 0` en JavaScript?

- A** Una excepción de división
 - B** Infinity
 - C** NaN
-

34 ¿Qué produce `0 / 0`?

- A** Infinity
 - B** NaN
 - C** 0
-

35 ¿Por qué `Number.isNaN("hoLa")` es más confiable que `isNaN("hoLa")` para detectar NaN?

- A** Convierte el string a número antes de comprobar
 - B** Devuelve true para cualquier string no numérico
 - C** No convierte el string a número antes de comprobar si es NaN
-

36 ¿Qué devuelve `Object.is(0, -0)` ?

- A** false
 - B** true
 - C** TypeError
-

37 ¿Por qué `0.1 + 0.2` puede no ser exactamente 0.3?

- A** Number guarda los decimales con solo dos cifras
 - B** El motor redondea el resultado a 16 decimales
 - C** Muchos decimales no tienen representación exacta en binario
-

38 ¿Qué comprueba `Number.isSafeInteger(valor)` ?

- A** Que es un número finito y no NaN
 - B** Que es un entero representable sin pérdida
 - C** Que el valor entra en un entero de 32 bits
-

39 ¿Qué devuelve `Number("42px")` ?

- A** NaN
 - B** 42
 - C** "42"
-

40 ¿Qué devuelve `parseInt("42px", 10)` ?

- A** NaN
 - B** 42
 - C** 0
-

41 ¿Qué valores dan `Math.floor(-3.2)` y `Math.trunc(-3.2)` ?

- A** -3 y -4
 - B** -3 y -3
 - C** -4 y -3
-

42 ¿Qué tipo devuelve `(12.5).toFixed(2)` ?

- A** string
 - B** number
 - C** bigint
-

43 ¿Qué ocurre con `10n + 2` ?

- A** Devuelve 12n
 - B** Devuelve 12 como number
 - C** Lanza TypeError
-

El tipo `string` y Unicode

Preguntas 44 a 58

[Volver al índice](#)

44 ¿Qué comillas permiten insertar expresiones dentro de una cadena?

- A** Las comillas invertidas, con la forma `${expresión}`
 - B** Las comillas simples y dobles
 - C** Las comillas dobles con `${}`
-

45 ¿Qué representa `\n` dentro de una cadena JavaScript?

- A** Una barra invertida seguida de una n
 - B** Un salto de línea
 - C** El final de la cadena
-

46 Después de `const s = "casa"; s[0] = "C";`, ¿qué valor tiene `s`?

- A** "Casa"
 - B** "casa"
 - C** "C"
-

- 47** Si se llama a `original.trim()` sin guardar el resultado, ¿qué pasa con el string `original` ?
- A** Pierde los espacios en el mismo string
 - B** Cambia solo si tenía espacios en los extremos
 - C** No cambia; trim devuelve una cadena nueva sin los espacios
-

- 48** ¿Qué devuelve `"JavaScript".at(-1)` ?
- A** "t"
 - B** "J"
 - C** undefined
-

- 49** ¿Qué devuelve `"JavaScript".slice(0, 4)` ?
- A** "JavaS"
 - B** "Java"
 - C** "Script"
-

- 50** ¿Qué método de string responde directamente si una frase contiene un texto dado?
- A** split
 - B** startsWith
 - C** includes
-

51 ¿Qué devuelve `"uno dos uno".replaceAll("uno", "1")` ?

- A** "1 dos 1"
 - B** "1 dos uno"
 - C** "uno dos 1"
-

52 ¿Qué devuelve `"a,b,c".split(",")` ?

- A** La cadena "abc"
 - B** Un array con una sola cadena "a,b,c"
 - C** Un array con "a", "b" y "c"
-

53 ¿Qué expresión convierte `valor` en texto sin fallar aunque `valor` sea `null` ?

- A** `valor.toString()`
 - B** `String(valor)`
 - C** `valor.join()`
-

54 ¿Qué valor tiene `"😊".length` ?

- A** 2 unidades UTF-16
 - B** 1, porque es un solo carácter visible
 - C** 4 puntos de código
-

55 ¿Qué produce `Array.from("A😊B")` ?

- A** ["A", dos mitades de emoji, "B"]
 - B** ["A😊B"]
 - C** ["A", "😊", "B"]
-

56 ¿Qué herramienta divide un texto en caracteres tal como los ve una persona (grafemas), incluidos los emoji compuestos?

- A** `Array.from` sobre la cadena
 - B** `Intl.Segmenter`
 - C** normalizar con la forma NFC
-

57 Dos cadenas se ven iguales, pero están escritas con distintas secuencias Unicode, por ejemplo «é» como un solo carácter o como «e» más un acento. ¿Qué hay que hacer para poder compararlas?

- A** Normalizar ambas a NFC
 - B** Aplicar `trim` a una sola
 - C** Comparar sus longitudes UTF-16
-

58 ¿Qué herramienta ordena alfabéticamente nombres en español respetando la ñ y los acentos?

- A** Intl.Collator configurado para es
 - B** El operador menor que sin configuración
 - C** Comparar la longitud de los strings
-

undefined, null y la ausencia de datos

Preguntas 59 a 71

[Volver al índice](#)

59 ¿Qué valor tiene `pendiente` tras `let pendiente;` ?

- A** `null`
 - B** `""` (cadena vacía)
 - C** `undefined`
-

60 ¿Qué valor se asigna a propósito para indicar que un dato no existe o no tiene valor?

- A** `undefined`
 - B** `null`
 - C** `NaN`
-

61 ¿Qué devuelve `typeof null` ?

- A** `"null"`
 - B** `"undefined"`
 - C** `"object"`
-

62 En un programa, `cantidad` puede valer 0 legítimamente. ¿Por qué `if (!cantidad)` no sirve para detectar que falta el dato?

- A** Porque también entra cuando cantidad vale 0, que es válido
 - B** Porque no entra cuando cantidad es undefined
 - C** Porque `!cantidad` lanza error si falta el dato
-

63 ¿Qué devuelve `false ?? true` ?

- A** true
 - B** false
 - C** undefined
-

64 Si `usuario` es null, ¿qué produce `usuario?.direccion?.ciudad` ?

- A** null
 - B** undefined
 - C** TypeError
-

65 Con `function f(x = 3) { return x; }`, ¿cuándo se aplica el 3?

- A** Cuando el argumento es null
 - B** Cuando el argumento es falsy
 - C** Cuando falta o es undefined
-

66 ¿Qué diferencia hay entre los objetos `{}` y `{ valor: undefined }`?

- A** Solo el segundo tiene la propiedad `valor` como propia
 - B** Leer `.valor` da `undefined` solo en el primero
 - C** `JSON.stringify` los convierte en textos distintos
-

67 ¿Qué diferencia hay entre `[1, , 3]` y `[1, undefined, 3]` en la posición 1?

- A** Solo el segundo tiene esa posición presente
 - B** Solo el primero tiene esa posición presente
 - C** Ambos tienen exactamente la misma estructura
-

68 ¿Qué devuelve `[10, 20].indexOf(50)` si no encuentra el valor?

- A** `undefined`
 - B** `null`
 - C** `-1`
-

69 ¿Qué ocurre con una propiedad cuyo valor es `undefined` al serializar un objeto con `JSON.stringify`?

- A** Se escribe como `null`
 - B** Se omite
 - C** Se escribe como `"undefined"`
-

70 Una función devuelve `null` en tres situaciones distintas: todavía no hay datos, la búsqueda no encontró nada o hubo un error. ¿Qué representación distingue claramente cada caso?

- A** Un objeto con una propiedad estado que nombre cada caso
 - B** Un booleano que indique si hubo resultado
 - C** Distinguir null de undefined para cada caso
-

71 ¿Qué se gana al normalizar (limpiar y dar un formato único a) los datos externos apenas se reciben?

- A** La entrada deja de requerir validación posterior
 - B** Todo el programa usa datos con la misma forma
 - C** JSON.stringify conserva los campos undefined
-

Conversión y coerción de tipos

Preguntas 72 a 86

[Volver al índice](#)

72 ¿Qué expresión convierte `textoCantidad` a número de forma explícita?

- A** `textoCantidad * 1`
 - B** `textoCantidad.valueOf()`
 - C** `Number(textoCantidad)`
-

73 ¿Por qué `"6" - 1` produce 5?

- A** La resta solo opera con números y convierte el string a número
 - B** La resta concatena y luego convierte a número
 - C** JavaScript interpreta "6" como un literal numérico
-

74 ¿En qué orden conviene procesar un dato que llega de afuera, como un formulario o un archivo?

- A** Convertir, usar y validar al mostrar el resultado
 - B** Normalizar, convertir y validar antes de usarlo
 - C** Validar el texto crudo y convertir al final
-

75 ¿Qué devuelve `String(null)` ?

- A** `null`
 - B** `TypeError`
 - C** `"null"`
-

76 ¿Qué devuelve `Number("")` y por qué requiere cuidado en un campo obligatorio?

- A** 0; podría aceptarse una omisión como si fuera un cero
 - B** NaN; la omisión se detecta sin más control
 - C** undefined; se distingue fácilmente de cero
-

77 ¿Por qué `Number([])` devuelve 0?

- A** `Number` devuelve la longitud del array vacío
 - B** Se convierte primero en `""` y luego en 0
 - C** Un array vacío es falsy y false vale 0
-

78 Para el texto `"42px"`, ¿qué diferencia hay entre `Number` y `parseInt` ?

- A** Ambos dan `42`
 - B** `Number` da `42` y `parseInt` da `NaN`
 - C** `Number` da `NaN` y `parseInt` da `42`
-

79 ¿Por qué `Boolean("false")` no sirve para interpretar esa palabra?

- A** Devuelve false, pero no reconoce "FALSE"
 - B** Devuelve true: la cadena no está vacía
 - C** Lanza `TypeError` al no ser un booleano
-

80 Un entero muy grande llega como texto, en la variable `texto`. ¿Cómo se convierte a `bigint` sin perder dígitos?

- A** `BigInt(texto)`
 - B** `BigInt(Number(texto))`
 - C** `BigInt(parseInt(texto))`
-

81 ¿Qué resultado produce `"8" * "3"`?

- A** "83"
 - B** NaN
 - C** 24
-

82 ¿Qué produce `1 + 2 + "3"`?

- A** "33"
 - B** "123"
 - C** 6
-

83 ¿Por qué `"20" < "3"` puede ser true?

- A** Ambos se convierten a número antes de comparar
 - B** Se compara la longitud de las cadenas
 - C** Ambos son strings y se comparan carácter por carácter
-

84 ¿Qué devuelve `0 === false` ?

- A** true
 - B** false
 - C** undefined
-

85 Cuando JavaScript necesita convertir un objeto a un valor primitivo, ¿qué método usa primero si el objeto lo define?

- A** Symbol.toPrimitive
 - B** Object.prototype.valueOf
 - C** toString
-

86 Un formulario acepta números con coma decimal, como `"3,5"`. ¿Qué hace falta además de llamar a `Number` ?

- A** Reemplazar la coma por punto y nada más
 - B** Definir el formato aceptado y normalizarlo
 - C** Usar `parseFloat`, que ya acepta la coma
-

El tipo `symbol` y los protocolos de JavaScript

Preguntas 87 a 101

[Volver al índice](#)

87 ¿Qué devuelve `Symbol("id") === Symbol("id")`?

- A** `true`
 - B** `undefined`
 - C** `false`
-

88 En `Symbol("interno")`, ¿para qué sirve el texto `"interno"`?

- A** Hace iguales los símbolos con ese texto
 - B** Convierte el símbolo en clave string
 - C** Ayuda a depurar; no define identidad
-

89 Un objeto tiene la propiedad string `id` y otra bajo `Symbol("id")`. ¿Qué relación guardan?

- A** La segunda sobrescribe la primera
 - B** Son claves distintas
 - C** Ambas se enumeran con el mismo nombre
-

90 ¿Qué ocurre con una clave `symbol` al llamar `Object.keys` sobre un objeto?

- A** No aparece en el resultado
 - B** Aparece convertida a string
 - C** Se elimina del objeto
-

91 ¿Qué función devuelve las claves `symbol` propias de un objeto?

- A** `Object.getOwnPropertyNames`
 - B** `Object.entries`
 - C** `Object.getOwnPropertySymbols`
-

92 ¿Copia `{ ...obj }` las propiedades de `obj` cuya clave es un `symbol`?

- A** No, se omiten las claves `symbol`
 - B** Sí
 - C** Solo si la clave se creó con `Symbol.for`
-

93 ¿Qué devuelve `Symbol.for("curso") === Symbol.for("curso")` ?

- A** `true`
 - B** `false`
 - C** `TypeError`
-

94 ¿Qué devuelve `Symbol.keyFor(s)` si `s` se creó con `Symbol()`, fuera del registro global?

- A** Su descripción
 - B** undefined
 - C** La cadena "local"
-

95 ¿Qué expresión convierte un symbol en texto sin lanzar error?

- A** `String(simbolo)`
 - B** `"id=" + simbolo`
 - C** `Number(simbolo)`
-

96 ¿Qué símbolo especial debe usar un objeto como nombre de un método para poder recorrerse con `for...of`?

- A** `Symbol.hasInstance`
 - B** `Symbol.toStringTag`
 - C** `Symbol.iterator`
-

97 El método `Symbol.iterator` de un objeto devuelve un iterador. ¿Qué método debe tener ese iterador?

- A** `value`, que devuelve el elemento actual
 - B** `next`, que devuelve `value` y `done`
 - C** `done`, que indica si quedan elementos
-

98 ¿Qué símbolo especial busca `for await...of` para recorrer valores que llegan de forma asincrónica?

- A** `Symbol.isConcatSpreadable`
 - B** `Symbol.species`
 - C** `Symbol.asyncIterator`
-

99 ¿Qué tipo de valor debe devolver un método `Symbol.toPrimitive`?

- A** Un valor primitivo
 - B** El propio objeto
 - C** Un string con la descripción
-

100 Si un objeto define `Symbol.toStringTag`, ¿qué cambia?

- A** El nombre que muestra `typeof`
 - B** El texto de `Object.prototype.toString`
 - C** La clave con que lo guarda `JSON.stringify`
-

101 Un objeto tiene índices numéricos y `length`, como un array. ¿Qué símbolo hace que `concat` lo expanda elemento por elemento?

- A** `Symbol.hasInstance`
 - B** `Symbol.keyFor`
 - C** `Symbol.isConcatSpreadable`
-

Arrays y matrices

Preguntas 102 a 125

[Volver al índice](#)

102 ¿Cuándo es un array la mejor estructura para guardar una colección?

- A** Cuando importan el orden y las posiciones
 - B** Cuando cada clave es un objeto arbitrario
 - C** Cuando los duplicados deben desaparecer
-

103 ¿Qué diferencia hay entre `Array.of(3)` y `Array(3)` ?

- A** `Array.of(3)` es `[3]`; `Array(3)` tiene tres huecos
 - B** Ambos crean un array con tres huecos vacíos
 - C** El primero tiene tres huecos; el segundo contiene 3
-

104 ¿Qué expresión comprueba de forma confiable si `x` es un array?

- A** `typeof x === "array"`
 - B** `x instanceof Object`
 - C** `Array.isArray(x)`
-

105 Con `const a = ["rojo", "azul"]`, ¿qué devuelve `a.at(-1)` ?

- A** "rojo"
 - B** "azul"
 - C** undefined
-

106 ¿Qué ocurre al asignar `a.length = 1` a `a = [1, 2, 3]` ?

- A** Se descartan los elementos 2 y 3 y la longitud pasa a 1
 - B** Los elementos 2 y 3 quedan ocultos pero existen
 - C** Lanza `TypeError: length es de solo lectura`
-

107 ¿Qué devuelve `1 in [1, , 3]` ?

- A** true
 - B** false
 - C** undefined
-

108 ¿Qué devuelve `[10, 20, 30, 40].slice(1, 3)` ?

- A** [20, 30, 40]
 - B** [10, 20, 30]
 - C** [20, 30]
-

109 ¿Qué devuelve `push` al agregar un elemento a un array?

- A** El elemento agregado
 - B** Una copia del array
 - C** La nueva longitud
-

110 ¿Qué devuelve `splice` además de modificar el array original?

- A** Una copia del array original antes del cambio
 - B** Un array con los elementos eliminados
 - C** La nueva longitud del array
-

111 ¿Qué expresión devuelve `true` porque encuentra `NaN` dentro del array?

- A** `[NaN].includes(NaN)`
 - B** `[NaN].indexOf(NaN) >= 0`
 - C** `NaN === NaN`
-

112 ¿Qué devuelve `find` si ningún elemento cumple la condición?

- A** `undefined`
 - B** `-1`
 - C** Un array vacío
-

113 ¿Qué bucle recorre directamente los valores de un array?

- A** for...in
 - B** for...of
 - C** switch
-

114 ¿Qué método produce un array nuevo con un resultado por elemento?

- A** forEach
 - B** pop
 - C** map
-

115 `filter` devuelve un array nuevo con algunos objetos del array original. ¿Qué relación tienen esos objetos con los del original?

- A** Son los mismos objetos: filter copia solo las referencias
 - B** Cada objeto se copia en forma superficial
 - C** Se copian solo los objetos modificados
-

116 ¿Por qué conviene pasar un valor inicial a `reduce`?

- A** Hace que reduce recorra el array de derecha a izquierda
 - B** Fija el acumulador y cubre el array vacío
 - C** Evita que el callback reciba el índice
-

117 ¿Qué devuelve `forEach`?

- A** Un array transformado
 - B** El último elemento
 - C** undefined
-

118 ¿Qué resultado produce el siguiente código?

```
const datos = [1, [2, [3]]];  
datos.flat(2);
```

- A** Un array plano con 1, 2 y 3
 - B** Un array con el 3 todavía anidado
 - C** Un array con el 1 y el 2 anidados
-

119 ¿Por qué `[2, 10, 1].sort()` no da necesariamente el orden numérico esperado?

- A** sort sin comparador ordena de mayor a menor
 - B** sort ordena según el orden de inserción original
 - C** Sin comparador, sort convierte a string y compara texto
-

120 Después de `const copia = [...original]`, donde `original` contiene objetos, ¿qué comparten ambos arrays?

- A** El mismo array exterior, con otro nombre
 - B** Las referencias a los objetos internos
 - C** Nada: los objetos se copian también
-

121 ¿Qué problema tiene crear una matriz de 3×3 con `Array(3).fill(Array(3).fill(0))` ?

- A** Las tres filas son el mismo array, así que cambiar una cambia todas
 - B** Las filas quedan con huecos en lugar de ceros
 - C** Solo la primera fila queda llena de ceros
-

122 Para un array con los valores 10, 20, 30, 40 y 50, ¿qué devuelve `slice(-2)` ?

- A** El array original reducido a 40 y 50
 - B** Un array nuevo con 40 y 50
 - C** Un array nuevo con 10 y 20
-

123 Para los valores 1, 2 y 3, ¿qué devuelven respectivamente `some(n => n > 2)` y `every(n => n > 2)` ?

- A** true y true
 - B** false y false
 - C** true y false
-

124 Tras ejecutar este código, ¿qué ocurre con `original` y `copia` ?

```
const original = [3, 1, 2];  
const copia = original.toSorted((a, b) => a - b);
```

- A** Ambos quedan ordenados de menor a mayor
 - B** original no cambia; copia queda ordenada
 - C** original queda ordenado; copia no cambia
-

125 ¿Por qué `Array.from({ length: 3 }, () => Array(3).fill(0))` evita el problema de las filas compartidas?

- A** Todas las filas usan la misma referencia a un único array
 - B** Convierte la matriz en un objeto sin índices
 - C** La función crea un array nuevo para cada fila
-

Objetos, propiedades y referencias

Preguntas 126 a 147

[Volver al índice](#)

126 ¿Qué estructura conviene para representar un alumno con campos fijos, como nombre y legajo?

- A** Un objeto
- B** Un Set
- C** Una cadena separada por comas

127 Si un objeto se crea con `{ 10: "diez" }`, ¿cómo aparece esa clave en `Object.keys`?

- A** Como el string "10"
- B** Como el número 10
- C** Como un symbol

128 Con `const campo = "precio"`, ¿qué expresión lee la propiedad `precio` de `producto` usando la variable?

- A** `producto.campo`
- B** `producto["campo"]`
- C** `producto[campo]`

129 ¿Qué efecto tiene `delete producto.stock` ?

- A** La conserva con valor undefined
 - B** Quita la propiedad stock
 - C** La convierte en null
-

130 ¿Qué diferencia hay entre `Object.hasOwn(obj, "x")` y `"x" in obj` ?

- A** in considera propiedades heredadas; hasOwn no
 - B** hasOwn considera heredadas; in no
 - C** Ambas revisan también las propiedades heredadas
-

131 `alumno.direccion` puede ser `null` o `undefined`. ¿Qué evita escribir `alumno.direccion?.ciudad` ?

- A** Validar cualquier campo obligatorio
 - B** Leer ciudad cuando direccion es null o undefined
 - C** Que ciudad valga undefined si falta
-

132 Con `const nombre = "Ana"`, ¿qué crea `{ nombre }` ?

- A** Un Set que contiene el string "Ana"
 - B** Un bloque de código que evalúa nombre
 - C** La propiedad nombre con valor "Ana"
-

133 ¿Por qué no conviene definir con una función flecha un método de objeto que usa `this` ?

- A** No puede devolver valores
 - B** No tiene `this` propio
 - C** Su `this` es el objeto donde se define
-

134 En un literal de objeto, ¿qué sintaxis usa el contenido de la variable `campo` como nombre de la clave?

- A** `{ campo: valor }`
 - B** `{ "campo": valor }`
 - C** `{ [campo]: valor }`
-

135 ¿Qué función devuelve pares [clave, valor] de propiedades propias enumerables?

- A** `Object.entries`
 - B** `Object.getOwnPropertyNames`
 - C** `Object.hasOwn`
-

136 ¿Qué devuelve `{ x: 1 } === { x: 1 }` ?

- A** `true`, porque sus campos coinciden
 - B** `undefined`, porque no se comparan objetos
 - C** `false`, porque son objetos distintos
-

137 `a` es un objeto. Después de `const b = a; b.nombre = "Ana";`, ¿qué valor tiene `a.nombre`?

- A** El valor anterior, porque b es una copia
 - B** El nuevo valor, porque a y b son el mismo objeto
 - C** undefined, porque b tomó la propiedad
-

138 `original` tiene una propiedad `usuario` que es un objeto. Después de `const copia = { ...original }`, ¿qué comparación da `true`?

- A** `copia.usuario === original.usuario`
 - B** `copia === original`
 - C** `copia.usuario === structuredClone(original.usuario)`
-

139 `base` tiene la propiedad `tema: "claro"`. ¿Qué valor de `tema` tiene el objeto `{ ...base, tema: "oscuro" }`?

- A** "claro"
 - B** "oscuro"
 - C** undefined
-

140 ¿Qué argumento modifica `Object.assign(destino, origen)`?

- A** origen
 - B** Ninguno: devuelve un objeto nuevo
 - C** destino
-

- 141** Al copiar un objeto que contiene otros objetos, ¿qué hace `structuredClone` que el spread `{ ...obj }` no hace?
- A** Puede crear objetos internos nuevos en lugar de compartirlos
 - B** Copia también las funciones de los objetos
 - C** Solo copia el primer nivel del objeto
-

- 142** Con `const { nombre: nombreCompleto } = persona`, ¿qué variable local se crea?
- A** nombre
 - B** nombreCompleto
 - C** persona.nombreCompleto
-

- 143** ¿Por qué `JSON.parse(JSON.stringify(obj))` no sirve para clonar cualquier objeto?
- A** Copia solo el primer nivel del objeto
 - B** Modifica el objeto original al serializarlo
 - C** Pierde undefined, funciones y fechas, y falla con ciclos
-

- 144** ¿Qué añade `Reflect.ownKeys(obj)` frente a `Object.keys(obj)` ?
- A** También claves symbol y propiedades no enumerables
 - B** Las claves heredadas del prototipo
 - C** Los valores junto con cada clave
-

145 Si se aplica `Object.freeze` a un objeto que contiene otro objeto, ¿qué cambio puede seguir siendo posible?

- A** Modificar el objeto interior, si no se congeló
 - B** Reasignar una propiedad directa con otro objeto
 - C** Agregar propiedades directas si no existían antes
-

146 En `const { idioma = "es" } = configuracion`, ¿cuándo se usa "es" ?

- A** Cuando idioma vale null o undefined
 - B** Cuando idioma es una cadena vacía
 - C** Cuando la propiedad falta o su valor es undefined
-

147 ¿Qué hace `Object.fromEntries` con una colección de pares clave y valor?

- A** Devuelve un Map con esos pares
 - B** Construye un objeto con esos pares como propiedades
 - C** Devuelve un array de claves
-

Map, Set y colecciones especializadas

Preguntas 148 a 162

[Volver al índice](#)

- 148** ¿Cuándo conviene usar un Map en lugar de un objeto común?
- A** Cuando se necesita convertir a JSON directo
 - B** Cuando importa el orden por posición numérica
 - C** Cuando las claves salen de los datos y pueden ser de cualquier tipo
-
- 149** ¿Puede un objeto usarse como clave de Map sin convertirse en string?
- A** No, Map convierte la clave en string
 - B** Sí
 - C** Solo si tiene una propiedad id
-
- 150** Un Map guarda el valor `undefined` en una clave. ¿Cómo se distingue esa clave de una que no existe?
- A** Con `mapa.has(clave)`
 - B** Con `mapa.get(clave) === undefined`
 - C** Comparando el tamaño con cero
-

151 ¿En qué orden recorre pares un Map?

- A** En orden alfabético de claves
 - B** En orden de inserción
 - C** En orden numérico de valores
-

152 ¿En qué orden recibe argumentos el callback de Map.forEach?

- A** clave, valor, mapa
 - B** índice, valor, mapa
 - C** valor, clave, mapa
-

153 Hay que buscar muchos alumnos por legajo dentro de un array. ¿Qué se gana al armar antes un Map con el legajo como clave?

- A** Busca por legajo directamente, sin recorrer el array cada vez
 - B** Ordena los alumnos por número de legajo
 - C** Evita que haya dos alumnos con el mismo nombre
-

154 ¿Qué tamaño tiene `new Set(["js", "node", "js"])`?

- A** 3
 - B** 1
 - C** 2
-

155 ¿Qué valor tiene `new Set([ { id: 1 }, { id: 1 } ]).size`?

- A** 1
 - B** 2
 - C** 0
-

156 Si se agrega a un Set un valor que ya contiene, ¿qué pasa con ese valor?

- A** Queda en su lugar original
 - B** Se mueve al final del Set
 - C** Se agrega otra copia al final
-

157 ¿Qué contiene la intersección de dos conjuntos?

- A** Los valores de uno u otro conjunto
 - B** Solo valores presentes en ambos
 - C** Valores presentes en exactamente uno
-

158 ¿Qué estructura conviene si importan el orden de los elementos y también sus repeticiones?

- A** Set
 - B** WeakSet
 - C** Array
-

159 Si los valores de un Map son objetos, ¿qué comparte `new Map(original)` con el Map original?

- A** Las referencias a esos objetos
 - B** El mismo Map, con otro nombre
 - C** Nada: los valores se clonan también
-

160 ¿Qué diferencia principal tiene un WeakMap respecto de un Map?

- A** No impide que el recolector libere sus claves objeto
 - B** Permite iterar todas sus claves
 - C** Acepta claves primitivas y objetos
-

161 ¿Para qué sirve WeakSet?

- A** Guardar valores primitivos sin duplicados
 - B** Marcar objetos sin impedir su liberación
 - C** Listar objetos en orden de inserción
-

162 ¿Qué método de Set indica si un valor pertenece al conjunto?

- A** `set.indexOf(valor)`
 - B** `set.get(valor)`
 - C** `set.has(valor)`
-

Estructuras de control

Preguntas 163 a 182

[Volver al índice](#)

- 163** ¿Qué hace más legible la condición de un `if` que expresa una regla importante?
- A** Un valor truthy sin comparar
 - B** Una condición con nombre que explique la regla
 - C** Una asignación dentro del `if`
-
- 164** Cada vez que se ejecuta `if (condición) { A } else { B }`, ¿qué está garantizado?
- A** Se ejecuta uno de los dos caminos
 - B** Se ejecutan A y B en orden
 - C** No se ejecuta ninguno aunque la condición sea válida
-
- 165** Una cadena de `if...else if` clasifica notas en dos grupos: 8 o más, y 6 o más. ¿Qué condición debe ir primero?
- A** La de 6 o más
 - B** Cualquiera, pues se evalúan ambas ramas
 - C** La de 8 o más
-

166 Una función empieza con `if (!pedido) return;`. ¿Qué se logra con esa guarda?

- A** Hace que todas las condiciones sean globales
 - B** Obliga a usar switch
 - C** Deja el camino válido con menos anidamiento
-

167 ¿Cuándo conviene anidar dos `if` en lugar de unir las condiciones con `&&`?

- A** Cuando una decisión solo tiene sentido dentro de la otra
 - B** Cuando ambas condiciones son independientes y triviales
 - C** Cuando hay que garantizar que se evalúen las dos condiciones
-

168 ¿Qué equivale a `!(tieneSaldo && hayStock)`?

- A** `!tieneSaldo && !hayStock`
 - B** `!tieneSaldo || !hayStock`
 - C** `tieneSaldo || hayStock`
-

169 ¿Cómo compara `switch` el valor con cada `case`?

- A** Con igualdad flexible (`= =`)
 - B** Con igualdad estricta
 - C** Con comparación estructural profunda
-

170 ¿En qué situación conviene usar `switch` ?

- A** Clasificar intervalos numéricos solapados
 - B** Repetir hasta que cambie una condición
 - C** Elegir según códigos fijos de un estado
-

171 ¿Qué pasa en `switch` si un `case` no termina con `break`, `return` o `throw`?

- A** Sigue ejecutando el `case` siguiente hasta encontrar un corte
 - B** Sale del `switch` al final del `case`
 - C** Se produce `SyntaxError` al analizar el código
-

172 ¿Para qué sirve encerrar entre llaves el código de cada `case` de un `switch` ?

- A** Para que cada `case` tenga su propio alcance
 - B** Para que el `break` sea opcional en ese `case`
 - C** Para que `case` compare con igualdad flexible
-

173 `intentos` empieza en 0. ¿Qué le falta a `while (intentos < 3) { ejecutar(); }` para terminar?

- A** Un `else` que termine el bucle
 - B** Incrementar `intentos` en el cuerpo
 - C** Declarar `intentos` con `const`
-

174 ¿Qué bucle ejecuta el cuerpo al menos una vez antes de evaluar su condición?

- A** while
 - B** for clásico
 - C** do...while
-

175 ¿En qué orden se ejecutan las partes de un **for** clásico?

- A** Inicio, condición, cuerpo, actualización, condición
 - B** Inicio, condición, actualización, cuerpo, condición
 - C** Inicio, cuerpo, condición, actualización, cuerpo
-

176 ¿Qué bucle recorre valores de un array o Set?

- A** for...in
 - B** switch
 - C** for...of
-

177 ¿Qué recorre **for...in** sobre un objeto?

- A** Los valores de las propiedades propias
 - B** Claves enumerables, también las heredadas
 - C** Las claves propias, incluidas las symbol
-

178 ¿Qué hace `break` dentro de un bucle?

- A** Salta solo a la vuelta siguiente
 - B** Reinicia el contador
 - C** Termina el bucle más cercano
-

179 ¿Qué hace `continue` dentro de un bucle?

- A** Pasa a la vuelta siguiente
 - B** Termina el bucle más cercano
 - C** Sale de todos los bucles anidados
-

180 ¿Qué permite `break etiqueta` en bucles anidados?

- A** Salir del bucle interior, como un `break` común
 - B** Salir directamente del bucle que tiene esa etiqueta
 - C** Volver al comienzo del bucle exterior
-

181 Un bucle recorre un array por índice y elimina elementos con `splice`. ¿Por qué puede saltarse elementos?

- A** El siguiente se corre al índice actual
 - B** `splice` deja un hueco en esa posición
 - C** `splice` reordena el resto del array
-

182 ¿Cuándo conviene usar el operador ternario `condición ? a : b` ?

- A** Al encadenar varias decisiones anidadas
 - B** Al ejecutar una acción sin valor de retorno
 - C** Al elegir un valor entre dos alternativas de forma breve
-

Errores y excepciones

Preguntas 183 a 200

[Volver al índice](#)

- 183** Dentro de un bloque, `JSON.parse` lanza un error porque el texto no es JSON válido. ¿Qué pasa con la sentencia siguiente de ese bloque?
- A** Se ejecuta con el valor undefined
 - B** No se ejecuta: la excepción corta el bloque y se propaga
 - C** Se ejecuta y el error se registra en consola
-
- 184** Si una sentencia dentro de `try` lanza un error, ¿dónde continúa la ejecución?
- A** Al comienzo del try
 - B** Directo al finally
 - C** Al catch correspondiente
-
- 185** ¿Por qué se recomienda lanzar `new Error(mensaje)` y no una cadena?
- A** Permite capturarlo con catch
 - B** Incluye nombre, mensaje y pila
 - C** Hace que el error no detenga el programa
-

186 Una función `dividir` no admite divisor cero y, si lo recibe, hace `throw new RangeError(...)`. ¿Qué expresa ese error?

- A** Que la entrada incumple el contrato de esa función
 - B** Que JavaScript lanza al dividir un number por cero
 - C** Que cero se convierte en null
-

187 Una función busca un alumno, y que no exista es un resultado habitual. ¿Qué conviene que devuelva si no lo encuentra?

- A** Lanzar una excepción al no encontrarlo
 - B** Un objeto alumno con campos vacíos
 - C** Un resultado documentado como null
-

188 ¿Qué error lanza JavaScript cuando una operación recibe un valor de un tipo que no corresponde?

- A** TypeError
 - B** RangeError
 - C** URIError
-

189 ¿Qué error lanza `JSON.parse` si el texto está mal formado?

- A** ReferenceError
 - B** SyntaxError
 - C** AggregateError
-

190 Una función de stock lanza un error que incluye las propiedades `disponible` y `requerido`. ¿Qué ventaja tiene eso?

- A** Hace innecesario el mensaje de error
 - B** Permite decidir con datos, no con el texto
 - C** Hace que el error no necesite capturarse
-

191 Se captura un error técnico y se lanza otro, más comprensible para la aplicación. ¿Qué propiedad del nuevo error guarda el error original?

- A** `stackLength`
 - B** `responseText`
 - C** `cause`
-

192 ¿Por qué conviene evitar un único `try` que envuelva el parseo, la validación, el guardado y la notificación?

- A** Se vuelve difícil saber qué etapa falló y cómo recuperarse
 - B** Impide que finally se ejecute
 - C** Hace más lento el código del `try`
-

193 Un `catch` recibe un error que no sabe manejar. ¿Cómo conviene relanzarlo para conservar su tipo y su pila?

- A** `throw new Error(error.message)` sin causa
 - B** `throw error`
 - C** `return false`
-

194 ¿Cuándo se ejecuta el bloque `finally` ?

- A** Siempre: tras un éxito, un return o una excepción del try
 - B** Solo cuando el try lanza una excepción
 - C** Solo si el try no terminó con return
-

195 ¿Qué riesgo tiene escribir `return` dentro de un bloque `finally` ?

- A** Impide que finally termine de limpiar
 - B** Relanza la excepción que se había capturado
 - C** Puede tapar el return o el error del try
-

196 Dentro de un `try` se llama a una función que devuelve una promesa, sin `await` . Si la promesa se rechaza después, ¿por qué el `catch` no lo captura?

- A** `catch` no puede recibir objetos Promise
 - B** El try ya terminó cuando llega el rechazo
 - C** El rechazo se convierte en undefined sin error
-

197 Después de `const response = await fetch(url)`, ¿qué hay que comprobar para detectar una respuesta 404?

- A** `response.ok` o `response.status`
 - B** Si la promesa de `fetch` fue rechazada
 - C** Si `response.json()` devuelve `null`
-

198 ¿Qué devuelve `Promise.allSettled` cuando algunas tareas fallan?

- A** Solo los valores de las tareas exitosas
 - B** El primer rechazo, como `Promise.all`
 - C** Un objeto por tarea con su estado: `fulfilled` o `rejected`
-

199 Si la aplicación falla por un error interno, ¿qué conviene mostrarle a quien la usa?

- A** El mensaje técnico original del error
 - B** Un mensaje claro, sin detalles internos
 - C** El código de error y la pila resumida
-

200 En una tienda, quedarse sin stock es un caso frecuente. ¿Cómo conviene que la función que procesa la compra informe ese caso?

- A** Lanzar un Error y capturarlo arriba
 - B** Devolver undefined sin explicación
 - C** Un resultado con ok false y motivo
-

Funciones

Preguntas 201 a 226

[Volver al índice](#)

201 ¿Qué diferencia hay entre `sumar` y `sumar(2, 3)` ?

- A** Uno nombra la función; el otro la ejecuta con 2 y 3
 - B** Ambos llaman; el primero sin argumentos
 - C** El primero es un string con el nombre
-

202 En `function presentar(nombre) {}`, ¿qué es `nombre` ?

- A** Un argumento de la llamada
 - B** Un parámetro de la función
 - C** Una variable global implícita
-

203 Una función se llama sin pasarle un argumento, y ese parámetro no tiene valor predeterminado. ¿Qué valor tiene el parámetro?

- A** undefined
 - B** null
 - C** ReferenceError
-

204 ¿Se puede llamar a `duplicar()` en una línea anterior a su declaración `function duplicar(n) {...}`, dentro del mismo alcance?

- A** No, lanza `ReferenceError`
 - B** Solo si está en modo estricto
 - C** Sí
-

205 ¿Se puede llamar a una función guardada con `const f = () => ...` en una línea anterior a esa declaración?

- A** Sí, se eleva igual que una declaración completa
 - B** No, sigue las reglas de `const`
 - C** Solo si no recibe argumentos
-

206 Una función flecha sin llaves debe devolver un objeto literal. ¿Con qué hay que rodear el objeto?

- A** Corchetes de array
 - B** Comillas
 - C** Paréntesis
-

207 Dentro de un método común se define una función flecha. ¿Qué `this` usa la flecha?

- A** El mismo `this` que tiene el método que la contiene
 - B** Uno nuevo creado por la flecha
 - C** `globalThis`, en modo no estricto
-

208 Un método usa `this`. ¿Qué puede pasar si se guarda ese método en una variable y se lo llama desde ella?

- A** El objeto se clona
 - B** El método se convierte en generador
 - C** Puede perder el receptor original
-

209 ¿Qué hace `persona.saludar.bind(persona)`?

- A** Llama a saludar con `this = persona`
 - B** Crea una función nueva con `this` fijado en `persona`
 - C** Cambia saludar en el propio objeto
-

210 Con `function f(x = 3) { return x; }`, ¿con qué argumento `f` devuelve 3?

- A** `undefined`
 - B** `null`
 - C** `""` (cadena vacía)
-

211 ¿Dónde debe ubicarse `...resto` en la lista de parámetros?

- A** Al comienzo
 - B** Al final
 - C** En cualquier posición
-

212 ¿Qué ventaja tiene el parámetro `rest` (`...args`) frente al objeto `arguments`?

- A** Un `this` propio dentro de las flechas
 - B** La cantidad de argumentos recibidos
 - C** Un array real de argumentos, también en flechas
-

213 La función `function f({ a, b }) {...}` se llama como `f(null)`. ¿Qué ocurre?

- A** Lanza `TypeError` al intentar desestructurar `null`
 - B** Usa los valores predeterminados
 - C** Desestructura un objeto vacío
-

214 Una función recibe un número y dentro le asigna el doble a su parámetro. ¿Cambia la variable que se pasó como argumento?

- A** Sí, porque se pasa por referencia
 - B** No
 - C** Solo si la función usa `return`
-

215 Una función recibe un objeto y modifica una de sus propiedades. ¿Qué ve después el código que la llamó?

- A** El cambio, porque ambos apuntan al mismo objeto
 - B** El objeto sin cambios, porque se copió
 - C** El cambio, solo si la función lo devuelve
-

216 Una función recibe un objeto y asigna un objeto nuevo a su parámetro. ¿A qué apunta la variable que se pasó como argumento?

- A** Pasa a apuntar al nuevo objeto
 - B** Se vuelve undefined
 - C** Permanece apuntando al objeto original
-

217 ¿Qué devuelve una función que solo hace console.log y no retorna?

- A** El texto impreso
 - B** La última variable local
 - C** undefined
-

218 ¿Qué pasa si se escribe `return` y el objeto literal que se quiere devolver empieza en la línea siguiente?

- A** Que el objeto se devuelva normalmente
 - B** Que la función devuelva undefined
 - C** Que se produzca un `SyntaxError`
-

219 Una función necesita devolver varios datos. ¿Qué estructura les da nombres claros?

- A** Un objeto con una propiedad por cada dato
 - B** Varios `return`, uno por cada dato
 - C** Un array con los datos en orden
-

220 Una función interior usa una variable exterior, y esa variable cambia después de crear la función. ¿Qué valor ve la función interior?

- A** Una copia del valor inicial
 - B** El valor que tenía al declarar la clausura
 - C** La variable, con su valor actual
-

221 ¿Qué devuelve siempre una función marcada `async`?

- A** Un valor síncrono sin envoltorio
 - B** Una promesa
 - C** Un generador
-

222 ¿Cuándo se ejecuta el código dentro de una función generadora?

- A** Cuando se llama a `next()`
 - B** Al llamar a la función generadora
 - C** Al declararse la función
-

223 Una función usa la variable `tasa` sin declararla adentro. ¿Dónde la busca JavaScript?

- A** En el objeto desde el que la llamaron
 - B** En el alcance del lugar donde se la llama
 - C** En los alcances del lugar donde fue definida la función
-

224 `crearContador()` devuelve funciones que modifican una variable local. Si se llama dos veces a `crearContador()`, ¿qué comparten los dos contadores?

- A** Nada: cada llamada tiene su propio estado
 - B** El mismo contador, porque la función es única
 - C** Solo el valor inicial del contador
-

225 ¿Qué muestra `funciones.map(fn => fn())` al final de este código?

```
const funciones = [];  
for (let i = 0; i < 3; i += 1) {  
  funciones.push(() => i);  
}
```

- A** Un array con 3, 3 y 3
 - B** Un array con 0, 1 y 2
 - C** Un array con 2, 2 y 2
-

226 Al documentar una función que recibe un callback, ¿qué conviene aclarar?

- A** Si lo ejecuta enseguida, varias veces o más tarde
 - B** Que lo ejecuta exactamente una vez y de forma inmediata
 - C** Que el callback no puede devolver un valor
-

Programación funcional y pipelines de datos

Preguntas 227 a 246

[Volver al índice](#)

227 ¿Qué caracteriza a una función de orden superior?

- A** Se llama a sí misma
 - B** Recibe o devuelve funciones
 - C** No tiene efectos secundarios
-

228 ¿Qué condiciones cumple una función pura?

- A** Mismo resultado, aunque modifique variables externas
 - B** Ningún efecto externo, aunque cambie el resultado
 - C** Siempre el mismo resultado y ningún efecto externo observable
-

229 Una función que calcula un impuesto lee la variable global `tasaActual`. ¿Por qué eso la vuelve impura?

- A** Leer una variable global la modifica
 - B** Toda multiplicación con decimales es impura
 - C** Su resultado depende de algo externo
-

230 En un programa hecho con funciones puras, ¿dónde conviene ubicar la lectura de archivos y la escritura de resultados?

- A** Dentro de cada transformación del pipeline
 - B** En los bordes del programa: al entrar y al salir
 - C** En una función auxiliar por cada dato
-

231 ¿En qué se diferencia `{ ...alumno, nota: 8 }` de hacer `alumno.nota = 8`?

- A** Crea una versión exterior nueva con nota 8
 - B** Clona profundamente cualquier objeto anidado
 - C** Borra todas las otras propiedades
-

232 Una función crea un Map dentro de su cuerpo, lo modifica y no toca sus argumentos. ¿Puede ser pura?

- A** No, cualquier mutación la vuelve impura
 - B** Solo si el Map se congela al terminar
 - C** Sí, si no hay efectos visibles afuera
-

233 ¿Cuántos elementos agrega `map` al resultado por cada elemento del array?

- A** Cero o una, según el callback
 - B** Una
 - C** Tantas como devuelva el callback
-

234 ¿Qué decide la función que se le pasa a `filter` ?

- A** Si se conserva cada elemento
 - B** El orden de inserción global
 - C** La cantidad exacta de llamadas futuras
-

235 En `reduce` , ¿qué papel cumple el valor inicial?

- A** Es el primer elemento que se procesa
 - B** Es el acumulador de la primera vuelta
 - C** Es el valor que se devuelve si falla
-

236 ¿Qué método de array deja de recorrer apenas encuentra un elemento que cumple la condición y devuelve un booleano?

- A** `every`
 - B** `forEach`
 - C** `some`
-

237 ¿Cuándo es `forEach` el método adecuado?

- A** Cuando solo importa el efecto por elemento
 - B** Cuando se necesita un array transformado
 - C** Cuando hay que cortar el recorrido antes
-

238 ¿Cuántos elementos puede aportar al resultado cada llamada de la función que recibe `flatMap` ?

- A** Cero, uno o varios elementos
 - B** Exactamente un elemento, como `map`
 - C** Un array anidado sin aplanar
-

239 ¿Qué método ordena un array devolviendo un array nuevo, sin modificar el original?

- A** `sort`
 - B** `toReversed`
 - C** `toSorted`
-

240 ¿Qué hacen, en orden, los pasos `filter` , `map` y `reduce` encadenados?

- A** Acumular, ordenar y borrar
 - B** Seleccionar, transformar y acumular
 - C** Crear un objeto, clonarlo y serializarlo
-

241 ¿Qué hace una función `pipe` que recibe varias funciones y las aplica de izquierda a derecha?

- A** Ejecuta todas las funciones sobre la entrada original sin relación
 - B** Vuelve cada función recursiva
 - C** Pasa la salida de una transformación a la siguiente
-

242 Si `esImpar = n => !esPar(n)`, ¿qué papel tiene `esPar`?

- A** Un predicado que se reutiliza
 - B** Una función de orden superior
 - C** Un callback que se ejecuta después
-

243 ¿Qué ocurre al ejecutar `array.filter(...).map(...)`?

- A** Nada se ejecuta hasta pedir el resultado
 - B** Cada una crea un array intermedio
 - C** Se combinan en un único recorrido
-

244 ¿Qué ventaja tiene usar un generador para producir una secuencia muy grande?

- A** Calcular todos los valores por adelantado
 - B** Ejecutarse en paralelo con el resto
 - C** Producir solo los valores consumidos
-

245 ¿Por qué `["10", "10", "10"].map(parseInt)` puede dar resultados inesperados?

- A** map pasa el índice y parseInt lo usa como base
 - B** map convierte cada string en número antes
 - C** parseInt necesita que la base sea un string
-

246 ¿Qué devuelve la función `n => { n * 2; }` al llamarla?

- A** El doble de n
 - B** undefined
 - C** Un objeto con el doble de n
-

Recursividad y árboles binarios

Preguntas 247 a 268

[Volver al índice](#)

247 ¿Qué crea cada llamada recursiva para guardar sus parámetros y variables locales?

- A** Una variable global nueva
 - B** Un marco propio en la pila
 - C** Un hilo de ejecución propio
-

248 ¿Qué hace el caso base de una función recursiva?

- A** Resuelve una entrada sin hacer otra llamada
 - B** Duplica la entrada para la llamada siguiente
 - C** Ordena los nodos del árbol
-

249 Cuando una función recursiva no está en el caso base y se llama a sí misma, ¿qué debe garantizar esa llamada?

- A** Reducir el problema a la mitad
 - B** Hacer al menos dos llamadas
 - C** Acercarse al caso base con una entrada más pequeña
-

250 En `factorial(n) = n * factorial(n - 1)`, ¿qué operación combina el resultado de la llamada recursiva?

- A** La multiplicación por n
 - B** La lectura de n sin cambio
 - C** La creación de un array
-

251 Para sumar un array de millones de elementos, ¿por qué conviene un bucle en lugar de recursión?

- A** Permite sumar sin variable acumuladora
 - B** Evita un marco de pila por elemento
 - C** Recorre el array en orden inverso
-

252 Un documento tiene secciones que pueden contener otras secciones, sin límite de profundidad. ¿Por qué una función recursiva es adecuada para recorrerlo?

- A** Las secciones forman una lista plana
 - B** La profundidad se conoce de antemano
 - C** Cada sección tiene la misma forma que la sección que la contiene
-

253 En un árbol binario, ¿qué indica que un nodo tenga `null` como hijo?

- A** Un subárbol vacío
 - B** Un nodo con valor cero
 - C** Una referencia al padre
-

254 En un árbol binario de búsqueda, ¿dónde van los valores menores que el nodo?

- A** En la rama que tenga menos nodos
 - B** En su subárbol menor
 - C** En el subárbol mayor, si hay lugar
-

255 ¿Por qué hay que decidir una regla para los valores repetidos en un árbol binario de búsqueda?

- A** Porque el árbol no admite duplicados
 - B** Para que el recorrido inorden sea más rápido
 - C** Para que todas las operaciones coincidan
-

256 ¿En qué orden visita los nodos un recorrido inorden de un árbol binario?

- A** Nodo, menor, mayor
 - B** Menor, mayor, nodo
 - C** Menor, nodo, mayor
-

257 ¿Qué visita primero un recorrido preorden de un árbol binario?

- A** El nodo antes de sus hijos
 - B** La hoja más profunda
 - C** El subárbol mayor antes de la raíz
-

258 ¿Cuándo procesa cada nodo un recorrido postorden de un árbol binario?

- A** Antes de ambos hijos
 - B** Después de ambos hijos
 - C** Entre el hijo menor y el mayor
-

259 ¿Qué ventaja tiene recorrer un árbol con un generador?

- A** Evita toda recursión en el recorrido
 - B** Entrega valores a medida que se piden
 - C** Recorre el árbol en varios hilos a la vez
-

260 Al buscar un valor en un árbol binario de búsqueda, después de compararlo con un nodo, ¿por cuántas ramas se sigue buscando?

- A** Una rama como máximo
 - B** Ambas ramas
 - C** Ninguna aunque el valor difiera
-

261 En una inserción recursiva en un árbol se escribe `nodo.menor = insertar(nodo.menor, valor)`. ¿Por qué se reasigna `nodo.menor` ?

- A** Para que la raíz pase a ser el nodo menor
 - B** Para borrar el subárbol menor anterior
 - C** insertar puede devolver un nodo nuevo donde antes había null
-

262 Al insertar un valor en un árbol sin modificar el original (inserción inmutable), ¿qué nodos comparte el árbol nuevo con el anterior?

- A** Los del camino de la raíz al nuevo nodo
 - B** Los de ramas que no cambiaron
 - C** Ninguno, se clona todo
-

263 Si los nodos no guardan ningún contador, ¿qué hace falta para contar los nodos de un árbol?

- A** Visitar todos los nodos
 - B** Consultar solo la raíz
 - C** Consultar solo el mínimo
-

264 ¿Cómo se encuentra el valor mínimo en un árbol binario de búsqueda?

- A** Bajando por la rama mayor
 - B** Comparando todas las hojas
 - C** Bajando por la rama menor
-

265 Al eliminar una hoja de un árbol binario, ¿qué valor ocupa su lugar en el nodo padre?

- A** La raíz original
 - B** Una copia de ambos hijos
 - C** null
-

266 En un árbol binario de búsqueda, ¿cómo puede eliminarse un nodo con dos hijos?

- A** Reemplazarlo por su sucesor y quitar el sucesor de su rama
 - B** Subir su hijo mayor a su lugar
 - C** Reemplazarlo por la raíz del árbol
-

267 Para verificar que un árbol es de búsqueda, ¿por qué no alcanza con comparar cada nodo con sus hijos directos?

- A** Los hijos pueden ser null
 - B** Un ancestro también impone límites
 - C** El orden también depende de la altura
-

268 ¿Qué pasa si se insertan valores ya ordenados en un árbol binario de búsqueda que no se balancea?

- A** La altura queda logarítmica
 - B** Se forma una cadena y buscar puede requerir n comparaciones
 - C** El árbol se balancea al insertar
-

Programación Asíncrona

Preguntas 269 a 281

[Volver al índice](#)

269 ¿Qué devuelve siempre una función declarada `async`, incluso si retorna un valor simple?

- A** Una promesa
 - B** Un valor síncrono sin envoltorio
 - C** Un iterador
-

270 Una función `async` ejecuta `await` sobre una promesa que todavía no se resolvió. ¿Qué se detiene?

- A** Todo el hilo de JavaScript
 - B** La función que llamó a la función `async`
 - C** El resto de esa función `async`
-

271 ¿Por qué se usa `await` dos veces en `const respuesta = await fetch(url); const datos = await respuesta.json();`?

- A** El segundo vuelve a pedir el recurso
 - B** Cada operación devuelve su propia promesa
 - C** `json()` necesita `await` por ser un método
-

272 ¿Qué hay que hacer para que una respuesta 404 de `fetch` se trate como un error?

- A** Revisar `respuesta.ok` o el status y lanzar un error
 - B** Esperar que `fetch` rechace la promesa
 - C** Envolver el `fetch` en `try/catch`
-

273 Un botón se deshabilita mientras dura una consulta. ¿En qué bloque conviene volver a habilitarlo, tanto si la consulta sale bien como si falla?

- A** `catch` solamente
 - B** El primer `await`
 - C** `finally`
-

274 ¿Cómo se ejecutan al mismo tiempo varias tareas asincrónicas independientes, esperando a que terminen todas?

- A** `await` en cada una, una tras otra
 - B** Un `forEach async` sin esperar
 - C** `Promise.all` sobre sus promesas
-

275 ¿Por qué `array.forEach(async x => { await ... })` no espera a que terminen las operaciones?

- A** `forEach` ignora las promesas que recibe
 - B** `forEach` no acepta funciones `async`
 - C** `await` dentro de `forEach` bloquea todo el bucle
-

276 Dos tareas independientes pueden fallar y se necesita conocer el resultado de cada una. ¿Qué conviene usar?

- A** `Promise.all` y revisar cada posición
 - B** `Promise.allSettled` y revisar cada estado
 - C** `Promise.race` y esperar la primera
-

277 ¿Qué ocurre si una de las promesas entregadas a `Promise.all` se rechaza?

- A** La promesa conjunta resuelve con `undefined` en esa posición
 - B** La promesa conjunta se rechaza
 - C** Las demás promesas se cancelan de inmediato
-

278 Si `pedirB(a)` necesita el resultado de `pedirA()`, ¿cómo deben esperarse esas operaciones?

- A** Iniciar ambas dentro de `Promise.all` sin pasar el resultado
 - B** Usar `forEach` para que B lea el futuro valor de A
 - C** Primero esperar A y luego iniciar B con ese resultado
-

279 Tres pedidos independientes tardan un segundo cada uno. ¿Cuánto tardan aproximadamente en serie y juntos con `Promise.all` ?

- A** Tres segundos en serie y uno juntos
 - B** Un segundo en serie y tres juntos
 - C** Tres segundos en ambos casos
-

280 Dentro de un `try`, se hace `await` de una promesa que se rechaza. ¿Qué ocurre?

- A** `await` la convierte en `undefined`
 - B** El `catch` la recibe como excepción
 - C** El error escapa del `try` por ser asíncronico
-

281 Se escribe `await` dentro de una función común y da error. ¿Qué falta?

- A** Declararla `async`
 - B** Marcarla como `function*`
 - C** Agregar `Promise.all` fuera de la función
-

Gestión de archivos con Node.js

Preguntas 282 a 312

[Volver al índice](#)

282 ¿Qué secuencia describe las capas que hay entre un archivo guardado y los datos que usa un programa?

- A** Extensión, clase CSS y DOM
 - B** Nombre, variable global y objeto JSON
 - C** Ruta, bytes, codificación, texto y formato
-

283 ¿Qué garantiza que un archivo se llame `datos.json` ?

- A** Que su contenido es JSON válido
 - B** Que `readFile` lo devuelve ya parseado como objeto
 - C** Nada: la extensión es solo una pista
-

284 ¿Qué entorno aporta módulos como `node:fs/promises` para acceder a archivos?

- A** Node.js
 - B** El núcleo ECMAScript sin entorno
 - C** CSS
-

285 ¿Qué módulo de Node para archivos se usa directamente con `async` y `await` ?

- A** `readFileSync`
 - B** `node:fs/promises`
 - C** Un atributo HTML
-

286 ¿Qué problema causa usar `readFileSync` en un servidor que atiende muchas solicitudes?

- A** Devuelve una promesa que nadie espera
 - B** Lee el archivo con otra codificación
 - C** Bloquea el hilo y demora a todas las demás solicitudes
-

287 ¿Qué extensión de archivo hace que Node lo trate como módulo ES sin configurar nada en `package.json` ?

- A** `.mjs`
 - B** `.css`
 - C** `.json`
-

288 `readFile("datos/entrada.txt")` usa una ruta relativa. ¿Desde qué carpeta la resuelve Node?

- A** El directorio del módulo
 - B** `process.cwd()`
 - C** La carpeta raíz del sistema
-

289 En un módulo ES, ¿qué dato permite armar una ruta relativa a la carpeta del propio archivo?

- A** `import.meta.url`
 - B** `process.pid`
 - C** `Buffer.byteLength`
-

290 ¿Qué función une partes de una ruta sin tener que escribir las barras a mano?

- A** `String.split`
 - B** `JSON.parse`
 - C** `path.join`
-

291 ¿Qué logra `mkdir("salidas", { recursive: true })`?

- A** Sobrescribir cualquier archivo llamado salidas
 - B** Crear también directorios padres faltantes
 - C** Vaciar salidas si ya existe
-

292 ¿Qué hace `writeFile` por defecto si el archivo destino ya existe?

- A** Lo reemplaza
 - B** Falla con `EEXIST`
 - C** Agrega al final
-

293 ¿Qué flag de `writeFile` hace que falle si el archivo ya existe, en lugar de reemplazarlo?

- A** `w`
 - B** `wx`
 - C** `a+`
-

294 ¿Qué operación añade texto al final sin borrar el contenido previo?

- A** `writeFile` con flag `w`
 - B** `unlink`
 - C** `appendFile`
-

295 ¿Qué devuelve `readFile` cuando no se indica codificación?

- A** Un string en UTF-8
 - B** Un objeto JSON parseado
 - C** Un Buffer de bytes
-

296 ¿Cuántos bytes ocupa el emoji 😊 en UTF-8, según `Buffer.byteLength` ?

- A** 2 bytes
 - B** 4 bytes
 - C** 1 byte
-

297 Un `TextDecoder` creado con `{ fatal: true }` recibe bytes que no son UTF-8 válido. ¿Qué hace?

- A** Lanza un error en lugar de insertar un reemplazo
 - B** Inserta el carácter de reemplazo
 - C** Omite los bytes inválidos
-

298 Al leer texto creado en distintos sistemas, ¿qué dos convenciones de fin de línea conviene contemplar?

- A** LF y CRLF
 - B** CR y TAB
 - C** LF y NUL
-

299 Al reemplazar un archivo importante, ¿qué técnica evita que quede a medio escribir si el programa se corta?

- A** Borrar el destino y escribirlo de nuevo
 - B** Escribir con el flag `a` sobre el destino
 - C** Escribir un archivo temporal y después renombrarlo
-

300 ¿Qué información agrega `readdir` cuando se usa con `{ withFileTypes: true }`?

- A** El contenido de cada archivo listado
 - B** Si cada entrada es archivo o carpeta
 - C** El tamaño y la fecha de cada entrada
-

301 ¿Qué diferencia hay entre `stat` y `lstat` cuando la ruta es un enlace simbólico?

- A** `stat` sigue el enlace; `lstat` no
 - B** `lstat` sigue el enlace; `stat` no
 - C** `lstat` es la versión síncrona de `stat`
-

302 Un programa quiere leer un archivo solo si existe. ¿Qué forma evita el problema de que el archivo cambie entre la comprobación y la lectura?

- A** Llamar `access` y asumir que nada cambiará
 - B** Comparar solo la extensión
 - C** Intentar `readFile` y manejar `ENOENT`
-

303 ¿Cómo se procesa línea por línea un archivo de texto muy grande sin cargarlo entero en memoria?

- A** `readFile` y después `split` por saltos
 - B** `createReadStream` combinado con `readline`
 - C** `readFileSync` con `encoding utf8`
-

304 Al leer un archivo con un stream, ¿por qué no conviene tratar cada fragmento (chunk) como una línea?

- A** Los chunks llegan desordenados
 - B** Un chunk puede cortar una línea
 - C** Un chunk trae solo un carácter
-

305 ¿Qué resuelve `pipeline` al conectar un stream de lectura con uno de escritura?

- A** Manejo de errores, cierre y contrapresión
 - B** Lectura y escritura en hilos separados del sistema
 - C** Carga completa previa para evitar cortes
-

306 Un programa abre un archivo con `open` y obtiene un `FileHandle`. ¿Dónde conviene cerrarlo para que se cierre aunque falle la lectura?

- A** Solo después de un éxito
 - B** Node lo cierra solo al terminar
 - C** En finally
-

307 Después de `JSON.parse`, ¿qué falta hacer antes de usar los datos en la aplicación?

- A** Validar estructura, tipos y rangos
 - B** Nada, si `JSON.parse` no lanzó
 - C** Volver a serializar con `stringify`
-

308 ¿Por qué `línea.split(",")` no basta para cualquier CSV?

- A** `split` no acepta una coma como separador
 - B** `split` pierde el último campo de la línea
 - C** Puede haber comas dentro de un campo entre comillas
-

309 Al leer un archivo, ¿qué códigos de error distinguen la falta de permisos de una ruta inexistente?

- A** `ENOENT` o `EEXIST`, frente a `EACCES`
 - B** `EACCES` o `EPERM`, frente a `ENOENT`
 - C** `EMFILE` o `EXDEV`, frente a `ENOTDIR`
-

310 Si `rename` falla con `EXDEV` al mover un archivo, ¿qué procedimiento permite completar el movimiento?

- A** Copiar, verificar la copia y borrar el origen
 - B** Borrar el origen primero y después copiar el archivo
 - C** Reintentar `rename` hasta que funcione
-

311 Si procesar miles de rutas con `Promise.all` causa `EMFILE`, ¿qué enfoque equilibra recursos y rendimiento?

- A** Leer todas las rutas en serie, de a una
 - B** Limitar cuántas operaciones corren al mismo tiempo con un pool
 - C** Convertir todas las rutas a URLs antes de leer
-

312 Antes de usar `rm` con `recursive: true`, ¿qué debe comprobarse sobre la ruta?

- A** Que tenga una extensión conocida
 - B** Que sea relativa, así no necesita validarse
 - C** Que esté dentro de una raíz permitida
-

Desarrollo web: del documento a una aplicación del clima

Preguntas 313 a 362

[Volver al índice](#)

313 ¿Cuál es la relación entre Internet y la Web?

- A** La Web funciona sobre Internet
- B** Son dos nombres de la misma red
- C** Internet funciona sobre la Web

314 En `https://ejemplo.com/apuntes/html.html`, ¿qué representa `/apuntes/html.html` ?

- A** El dominio
- B** La ruta
- C** La consulta

315 ¿Qué agrega escribir `<h1>El tiempo</h1>` en lugar de poner el texto sin etiqueta?

- A** Solo lo muestra más grande
- B** Lo convierte en un enlace
- C** Le da significado de título

316 ¿Con qué criterio se elige entre `h1` y `h2` para un título?

- A** Por el tamaño de letra que se quiere
 - B** Por la importancia visual del texto
 - C** Por su nivel dentro de la estructura del documento
-

317 ¿Qué diferencia práctica hay entre `class` e `id` en HTML?

- A** El `id` se reutiliza; la clase es única
 - B** La clase se reutiliza; el `id` es único
 - C** Solo el `id` puede usarse desde CSS
-

318 Si `strong` está dentro de `p` y `p` dentro de `section`, ¿qué relación tiene `strong` con `section`?

- A** Es descendiente de `section`
 - B** Es hijo directo de `section`
 - C** Es padre de `section`
-

319 ¿Cuál de estos elementos no necesita etiqueta de cierre en HTML?

- A** `p`
 - B** `section`
 - C** `input`
-

320 ¿Qué sucede con `<button disabled="false">` en HTML?

- A** Queda habilitado
 - B** Queda deshabilitado
 - C** Depende del navegador
-

321 ¿Para qué sirve `<!doctype html>` al inicio del documento?

- A** Activa el modo de estándares en el navegador
 - B** Declara el idioma del documento
 - C** Carga la hoja de estilos base
-

322 ¿Qué elemento semántico identifica el contenido principal de una página?

- A** span
 - B** link
 - C** main
-

323 ¿Qué debe describir el atributo `alt` de una imagen que aporta información?

- A** El título que aparece al pasar el mouse
 - B** Lo que la imagen aporta si no se ve
 - C** El nombre del archivo de la imagen
-

324 ¿Cuándo corresponde usar una tabla (`table`) en HTML?

- A** Cuando los datos son filas y columnas
 - B** Para ubicar el menú junto al contenido
 - C** Para alinear campos de un formulario
-

325 ¿Qué debe coincidir para que `<label for="ciudad">` señale un input?

- A** El valor for y el id del input
 - B** El valor for y su class
 - C** El texto de label y el placeholder
-

326 ¿Qué atributo identifica un campo en el envío nativo de un formulario?

- A** id
 - B** name
 - C** class
-

327 ¿Qué ventaja tiene un botón `submit` real frente a un `div` programado para funcionar como botón?

- A** Validación del servidor sin programarla
 - B** Conversión automática a JSON
 - C** Envío con Enter y acceso por teclado
-

328 ¿Qué limitación tiene el atributo `required` en un campo de texto?

- A** No valida que el texto tenga sentido
 - B** Rechaza cadenas de solo espacios
 - C** Valida también en el servidor
-

329 ¿Qué elemento HTML enlaza una hoja CSS externa desde head?

- A** `<script src="styles.css" type="text/css">`
 - B** `<link rel="stylesheet" href="styles.css">`
 - C** `<style src="styles.css">`
-

330 ¿Qué selecciona `.tarjeta` en CSS?

- A** El elemento con id tarjeta
 - B** Los elementos `<tarjeta>` del documento
 - C** Elementos con la clase tarjeta
-

331 ¿Qué selecciona `.tarjeta.destacado` ?

- A** Un descendiente destacado de tarjeta
 - B** Elementos con cualquiera de las dos clases
 - C** Un mismo elemento que tiene ambas clases
-

332 ¿Qué selecciona `.tarjeta p`?

- A** Solo párrafos que también tienen clase tarjeta
 - B** Párrafos descendientes de un elemento tarjeta
 - C** Todas las tarjetas que contienen cualquier texto
-

333 Tres reglas CSS asignan un color distinto al mismo párrafo, con los selectores `#estado`, `.aviso` y `p`. ¿Cuál tiene mayor especificidad?

- A** `#estado`
 - B** `.aviso`
 - C** `p`
-

334 Dos reglas CSS asignan la misma propiedad al mismo elemento, con igual especificidad. ¿Cuál se aplica?

- A** La que aparece después
 - B** La que tiene el selector más corto
 - C** La que aparece primero
-

335 Si se define en `body`, ¿qué propiedad heredan los párrafos que no tienen un valor propio?

- A** `margin`
 - B** `border-radius`
 - C** `font-family`
-

336 ¿Respecto de qué se calcula `1.5rem` ?

- A** Del ancho del viewport
 - B** Del tamaño de fuente del elemento raíz
 - C** Del tamaño de fuente del elemento padre
-

337 Con `width 300px`, `padding 20px` por lado y `border 2px` por lado en `content-box`, ¿cuál es el ancho hasta el borde?

- A** 300px
 - B** 344px
 - C** 324px
-

338 Con `box-sizing border-box` y `width 300px`, ¿qué incluye ese ancho?

- A** Contenido más márgenes exteriores
 - B** Solo contenido
 - C** Contenido, padding y bordes
-

339 ¿Qué diferencia hay entre `display none` y `visibility hidden`?

- A** `hidden` conserva el espacio; `none` no
 - B** `none` conserva el espacio; `hidden` no
 - C** Ninguno conserva el espacio ocupado
-

340 ¿Por qué conviene que el estilo `:focus-visible` se vea con claridad?

- A** Muestra qué control tiene el foco
 - B** Resalta el control bajo el mouse
 - C** Indica qué campos son obligatorios
-

341 ¿Qué consigue `var(--acento)` en una regla CSS?

- A** Lee una variable de JavaScript global
 - B** Usa el valor de la propiedad `--acento`
 - C** Define la propiedad `--acento`
-

342 En una fila flex con dirección row, ¿qué propiedad distribuye el espacio sobrante en el eje principal?

- A** `align-items`
 - B** `flex-direction`
 - C** `justify-content`
-

343 ¿Qué expresa `grid-template-columns: repeat(3, 1fr)` ?

- A** Tres filas de igual altura
 - B** Tres columnas de 1px cada una
 - C** Tres columnas que reparten el espacio en partes iguales
-

344 Un diseño muestra los elementos apilados, pensado primero para móvil. ¿Qué hace una media query con `min-width: 40rem` ?

- A** Aplica desde ese ancho en adelante
 - B** Aplica por debajo de ese ancho
 - C** Fija el ancho de la página en 40rem
-

345 ¿Quién aporta el objeto `document` a JavaScript en una página?

- A** La sintaxis básica ECMAScript
 - B** El navegador
 - C** La hoja CSS
-

346 ¿Qué efecto tiene el atributo `defer` en `<script src="app.js" defer>` , ubicado en el `head` ?

- A** Lo ejecuta apenas termina de descargarse
 - B** Lo descarga después de analizar el HTML
 - C** Lo ejecuta cuando el navegador terminó de analizar el HTML
-

347 ¿Por qué lo que muestra el panel Elementos de las herramientas del navegador puede diferir del archivo HTML original?

- A** Elementos muestra una versión minificada
 - B** El DOM se corrige y JavaScript lo cambia
 - C** CSS reescribe el HTML al aplicar estilos
-

348 ¿Qué devuelve `document.querySelector("#estado")` si no encuentra el elemento?

- A** null
 - B** Un elemento vacío
 - C** undefined
-

349 ¿Qué propiedad conviene usar para mostrar en la página un nombre que llegó de una API?

- A** innerHTML sin validación
 - B** textContent
 - C** outerHTML con la respuesta completa
-

350 ¿Por qué se escribe `boton.addEventListener("click", manejar)` y no `boton.addEventListener("click", manejar())` ?

- A** Debe devolver un elemento al registrarse
 - B** El navegador no acepta callbacks
 - C** Debe ejecutarse cuando ocurra el evento
-

351 ¿Qué hace `evento.preventDefault()` en un submit cancelable?

- A** Cancela el comportamiento nativo de envío
 - B** Detiene la propagación del evento
 - C** Vacía los campos del formulario
-

352 ¿Qué distingue `evento.target` de `evento.currentTarget` ?

- A** target es el origen; currentTarget, el del listener
 - B** target es el del listener; currentTarget, el origen
 - C** Son el mismo elemento en todos los casos
-

353 Una app del clima busca ciudades por nombre y muestra una lista de coincidencias para que la persona elija una. ¿Por qué no usa directamente la primera?

- A** Porque la API exige elegir una por país
 - B** Para consultar las coordenadas de la ciudad que se quiso buscar
 - C** Para mostrar el clima de todas a la vez
-

354 Una app arma la URL de búsqueda con el nombre de una ciudad que tiene espacios y tildes. ¿Qué aporta usar `URLSearchParams` ?

- A** Traduce el nombre de la ciudad al inglés
 - B** Garantiza que la red responda
 - C** Codifica correctamente los valores de la query
-

355 En una app del clima, al empezar una búsqueda nueva se oculta el resultado anterior y se borra la lista de ciudades. ¿Para qué?

- A** Para liberar la memoria que usó el navegador
 - B** Para que la API no reciba dos pedidos
 - C** Para no mostrar datos viejos como actuales
-

356 Una app valida la temperatura con

`Number.isFinite(temperatura)` antes de mostrarla. ¿Qué ventaja tiene esto cuando la temperatura es 0 °C?

- A** Descarta 0 por ser un valor falsy
 - B** Acepta 0 como temperatura válida
 - C** Rechaza temperaturas bajo cero
-

357 Una API entrega tiempo Unix en segundos. ¿Qué necesita `new Date` para ese instante?

- A** Milisegundos, por lo que se multiplica por 1000
 - B** Segundos sin conversión
 - C** Una cadena de zona horaria como único argumento
-

358 ¿Qué permite `<meta name="viewport" content="width=device-width, initial-scale=1.0">` en una página adaptable?

- A** Que el navegador genere las media queries
 - B** Que la página funcione sin CSS
 - C** Que el ancho de diseño acompañe al dispositivo móvil
-

359 Dentro de un formulario, ¿qué tipo de botón se usa para una acción que no debe enviarlo?

A `type="submit"`

B `type="button"`

C `type="search"`

360 Si una imagen es puramente decorativa, ¿qué valor de `alt` indica que no aporta contenido?

A `alt=""`

B Omitir el atributo alt

C `alt="imagen decorativa"`

361 En la cascada de CSS, ¿qué modifica `!important`?

A El número de identificadores en el selector

B El orden físico de las reglas en el archivo

C La etapa de importancia de la declaración

362 Una app muestra `Failed to fetch`. ¿Qué panel del navegador ayuda a revisar solicitudes, estados y tiempos?

A Elementos o Elements

B Red o Network

C Aplicación o Application

Arrow.js: del contador al inventario

Preguntas 363 a 374

[Volver al índice](#)

363 En Arrow.js, ¿qué hace `reactive({ nombre: "Tornillos", cantidad: 3 })` ?

- A** Hace observables sus propiedades
- B** Guarda el objeto en localStorage
- C** Convierte el objeto en HTML

364 En una plantilla de Arrow.js, ¿por qué se escribe ``${() => producto.cantidad}`` en lugar de ``${producto.cantidad}`` ?

- A** Así Arrow sabe qué actualizar al cambiar
- B** Así la plantilla se evalúa una sola vez
- C** Así cantidad se convierte en texto

365 En un inventario hecho con Arrow.js, `Contador` es un componente que muestra la tarjeta de un producto. ¿Qué recibe y qué devuelve?

- A** Recibe una plantilla; devuelve un producto
- B** Recibe un producto; devuelve una plantilla
- C** Recibe un id; devuelve HTML en texto

366 En Arrow.js, ¿por qué se escribe `Contador(p).key(p.id)` al mostrar una lista de productos?

- A** Ordena la lista por id
 - B** Reinicia cada contador al filtrar la lista
 - C** Identifica cada producto aunque se mueva
-

367 En un buscador hecho con Arrow.js, el campo tiene `.value="$({() => estado.busqueda})"` y `@input="$({e => estado.busqueda = e.target.value})"`. ¿Qué hace cada uno?

- A** `.value` muestra el estado; `@input` lo actualiza
 - B** `@input` muestra el estado; `.value` lo actualiza
 - C** Ambos actualizan el estado al escribir
-

368 En un inventario hecho con Arrow.js, un producto se oculta al filtrar por búsqueda y reaparece al borrar el texto. ¿Por qué conserva su cantidad?

- A** Cada búsqueda recrea los productos desde cero
 - B** `filter` devuelve los mismos objetos del estado
 - C** Arrow guarda la cantidad en `localStorage`
-

369 En un inventario hecho con Arrow.js, el componente `Formulario` crea su propio estado `borrador`. ¿Por qué?

- A** Para que el inventario cambie en cada tecla
 - B** Porque Arrow no permite estado fuera de componentes
 - C** Para no tocar el inventario hasta confirmar
-

370 ¿Por qué se aplica `Number` a la cantidad que se lee de un campo con `e.target.value`?

- A** Arrow convierte todo número en bigint
 - B** El input devuelve undefined si está vacío
 - C** El valor del campo llega como texto
-

371 En Arrow.js, ¿qué hace `estado.productos = estado.productos.filter(p => p.id !== id)`?

- A** Elimina todos los productos del mismo nombre
 - B** Reemplaza la lista por una sin ese producto
 - C** Oculta el producto sin cambiar la lista
-

372 En un inventario de Arrow.js cuyos datos viven solo en memoria, ¿qué pasa con los productos agregados al recargar la página?

- A** Se pierden y reaparecen los valores iniciales
 - B** Se restauran desde una base de datos integrada en Arrow
 - C** Se recuperan desde localStorage
-

373 En un inventario hecho con Arrow.js, ¿cómo entrega el componente `Formulario` los datos de un producto nuevo a la aplicación?

- A** Modifica directamente `estado.productos` mientras se escribe
 - B** Envía el borrador al CDN de Arrow.js
 - C** Llama a `acciones.onAgregar(datos)` al confirmar
-

374 En una tarjeta hecha con Arrow.js, el botón de restar tiene `disabled="$({) => producto.cantidad === 0}"`. ¿Qué pasa cuando la cantidad llega a cero?

- A** Queda deshabilitado mediante una lectura reactiva de la cantidad
 - B** Se oculta de la tarjeta
 - C** Sigue restando hasta números negativos
-

Cómo consumir una API REST

Preguntas 375 a 400

[Volver al índice](#)

375 ¿Cuáles de estos programas pueden actuar como cliente HTTP?

- A** Solo un navegador con interfaz gráfica
 - B** Un navegador, curl o fetch desde un programa
 - C** Un servidor web, pero no curl
-

376 HTTP es un protocolo sin estado (stateless). ¿Qué implica eso?

- A** El servidor recuerda los pedidos anteriores
 - B** No se pueden usar cookies ni cabeceras
 - C** Cada pedido debe incluir lo necesario para entenderse solo
-

377 En la URL `https://ejemplo.com/usuarios?id=7#perfil`, ¿qué parte no se envía al servidor?

- A** El fragmento
 - B** La ruta
 - C** La cadena de consulta
-

378 ¿Qué cabecera indica que el cuerpo del pedido está en formato JSON?

- A** Accept: application/json
 - B** Content-Type: application/json
 - C** Content-Encoding: application/json
-

379 En una API REST de usuarios, ¿qué URL representa al usuario con id 7?

- A** /obtenerUsuario?id=7 como única forma REST
 - B** /users/7
 - C** /users#7
-

380 ¿Qué método HTTP se usa normalmente para leer un recurso sin modificarlo?

- A** POST
 - B** DELETE
 - C** GET
-

381 ¿Qué diferencia hay entre los métodos **PATCH** y **PUT** ?

- A** PATCH cambia campos; PUT reemplaza todo
 - B** PUT cambia campos; PATCH reemplaza todo
 - C** PATCH crea recursos; PUT los modifica
-

382 Un `POST` falla por un error de red y se reintenta. ¿Por qué hay que tener cuidado?

- A** POST es idempotente, como PUT
 - B** Puede crear el recurso dos veces
 - C** El servidor rechaza POST repetidos
-

383 ¿Qué significan los códigos HTTP 201 y 204?

- A** 201 para recurso creado; 204 para éxito sin cuerpo
 - B** 201 para éxito sin cuerpo; 204 para recurso creado
 - C** 201 para redirección; 204 para validación fallida
-

384 ¿Qué diferencia hay entre los códigos HTTP 401 y 403?

- A** 401: no hay permiso; 403: falta autenticarse
 - B** 401: servidor caído; 403: recurso movido
 - C** 401: falta autenticarse; 403: no hay permiso
-

385 ¿Qué informa el código HTTP 429?

- A** Tiempo de espera agotado
 - B** Cuerpo JSON inválido
 - C** Demasiadas solicitudes
-

386 ¿Cómo deben escribirse las claves de un objeto en JSON?

- A** Comillas dobles
 - B** Comillas simples
 - C** Ninguna comilla
-

387 ¿Qué operación convierte un objeto JavaScript en texto JSON para enviarlo?

- A** JSON.parse
 - B** JSON.stringify
 - C** Response.json
-

388 JSONPlaceholder es una API de práctica. Se crea un recurso con **POST** y después se lo pide con **GET**. ¿Qué pasa?

- A** No lo encuentra: el POST se simula y no se guarda
 - B** Lo encuentra con el id devuelto
 - C** Lo encuentra hasta que se reinicia
-

389 En un archivo **.http** de la extensión REST Client, ¿qué separa un pedido del siguiente?

- A** Una etiqueta HTML
 - B** Un archivo distinto por pedido
 - C** ###
-

390 ¿Qué opción de `curl` muestra las cabeceras de la respuesta junto con el cuerpo?

- A** -d
 - B** -i
 - C** -O
-

391 ¿Qué espera cada uno de estos pasos: `await fetch(url)` y `await respuesta.json()` ?

- A** Primero el cuerpo; luego su conversión
 - B** Primero las cabeceras; luego el cuerpo
 - C** Primero la conexión; luego las cabeceras
-

392 Después de `await respuesta.json()` , ¿qué indica `respuesta.bodyUsed` ?

- A** true; el cuerpo ya se consumió
 - B** false; puede leerse de nuevo sin límite
 - C** undefined porque Response no tiene cuerpo
-

393 ¿Qué pasa con la promesa de `fetch` si el servidor responde 404?

- A** La promesa se rechaza con un error
 - B** Se resuelve y `respuesta.ok` es true
 - C** Se resuelve y `respuesta.ok` es false
-

394 Una app de consola hecha con Ink maneja los estados `"pidiendo"`, `"cargando"`, `"mostrando"` y `"error"`. ¿Qué indica el estado `"cargando"` ?

- A** Que la API ya devolvió el clima
 - B** Que la consulta está en curso y aún no hay respuesta
 - C** Que la consulta falló y se reintenta
-

395 En un pedido HTTP, ¿qué cabecera expresa el formato de respuesta que acepta el cliente?

- A** Content-Length
 - B** Location
 - C** Accept
-

396 Un servidor recibe JSON bien formado, pero un campo no supera la validación. ¿Qué código HTTP representa ese caso?

- A** 422
 - B** 304
 - C** 201
-

397 ¿Qué comunica un estado HTTP `304 Not Modified` ?

- A** Que el recurso se movió a otra URL
 - B** Que el servidor no reconoce el método
 - C** Que la copia en caché sigue valiendo
-

398 ¿Qué aporta `url.searchParams.set("ciudad", nombre)` al construir una URL?

- A** Codifica el valor para la query, incluso espacios y tildes
 - B** Convierte el nombre en coordenadas
 - C** Agrega el valor sin codificar a la URL
-

399 En la salida de `curl -v`, ¿qué distinguen las líneas que empiezan con `>` de las que empiezan con `<`?

- A** `>` es lo recibido; `<` lo enviado
 - B** `>` es lo enviado; `<` lo recibido
 - C** `>` son errores; `<` son avisos
-

400 Si una respuesta de `fetch` tiene estado `204`, ¿qué debe hacer una función que espera JSON antes de llamar a `respuesta.json()`?

- A** No llamar a `json()` y devolver null
 - B** Llamar a `json()`, que devuelve `{}`
 - C** Tratar 204 como un error
-